Министерство образования Республики Беларусь

Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Информационные сети. Основы безопасности»

ОТЧЁТ
к лабораторной работе №4
на тему
«ЗАЩИТА ОТ АТАКИ ПРИ УСТАНОВКЕ TCP-СОЕДИНЕНИЯ И ПРОТОКОЛОВ ПРИКЛАДНОГО УРОВНЯ»
БГУИР 6-05-0612-02 23

Выполнил студент группы 353503
СЕБЕЛЕВ Дмитрий Юрьевич

_____
(дата, подпись студента)

Проверил ассистент каф. информатики
ГЕРЧИК Артем Вадимович

_____
(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1 Постановка задачи 3
2 Выполнение работы 4
2.1 Защита от атаки Slowloris 4
2.2 Защита от атаки большими объёмами данных 4
2.3 Защита от атаки превышения частоты запросов 5
Вывод 7
Приложение А (Обязательное) Листинг программного кода сервера 8

1 ПОСТАНОВКА ЗАДАЧИ

Целью данной лабораторной работы является получение практических навыков в области защиты серверных приложений от атак, направленных на нарушение доступности сервиса. В рамках работы необходимо разработать защищённый HTTP-сервер, реализующий механизмы защиты от разных типов атак на этапе установки TCP-соединения и в рамках заданного протокола прикладного уровня.

2 ВЫПОЛНЕНИЕ РАБОТЫ

2.1 Защита от атаки Slowloris

Атака Slowloris заключается в следующем: злоумышленник устанавливает соединение с сервером и начинает отправку HTTP-запроса, но завершает её не полностью. Он отправляет только заголовки, указывая в поле Content-Length, что собирается передать определённый объём данных, однако передаёт данные очень медленно или не передаёт вовсе. При этом соединение остаётся открытым и занимает место в пуле доступных подключений сервера. Если таких «зависших» соединений накопится достаточно много, сервер не сможет обрабатывать запросы легитимных пользователей.

Для защиты от данной атаки в сервере были настроены временные ограничения: на чтение запроса отводится 5 секунд, на запись ответа — 10 секунд, а на ожидание следующего запроса в рамках keep-alive соединения — 15 секунд. Если клиент не укладывается в эти сроки, сервер автоматически разрывает соединение, освобождая ресурсы для других пользователей.

На рисунке 2.1 представлен результат работы защиты: после имитации атаки сервер разорвал соединение через 5 секунд.

```
velebes@arch ... (main)> go run cmd/server/main.go
Connection dropped for 127.0.0.1. Reason: read tcp 127.0.0.1:8089->127.0.0.1:52070: i/o timeout
Connection dropped for 127.0.0.1. Reason: read tcp 127.0.0.1:8089->127.0.0.1:52042: i/o timeout
Connection dropped for 127.0.0.1. Reason: read tcp 127.0.0.1:8089->127.0.0.1:52036: i/o timeout
Connection dropped for 127.0.0.1. Reason: read tcp 127.0.0.1:8089->127.0.0.1:52060: i/o timeout
Connection dropped for 127.0.0.1. Reason: read tcp 127.0.0.1:8089->127.0.0.1:52054: i/o timeout
```

```
velebes@arch ... (main)> go run cmd/test_slowloris/main.go
Starting Slowloris attack (5 concurrent connections)...

[Client 2] Connected, sent partial data ...
[Client 3] Connected, sent partial data ...
[Client 5] Connected, sent partial data ...
[Client 1] Connected, sent partial data ...
[Client 4] Connected, sent partial data ...
[Client 5] Server dropped the connection after 5.00490 seconds
[Client 2] Server dropped the connection after 5.00494 seconds
[Client 1] Server dropped the connection after 5.00464 seconds
[Client 3] Server dropped the connection after 5.00500 seconds
[Client 4] Server dropped the connection after 5.00461 seconds

Test completed. The server successfully defended against hanging sockets.
```

Рисунок 2.1 – Результат работы защиты от атаки Slowloris

2.2 Защита от атаки большими объёмами данных

Данная атака предполагает отправку на сервер запросов с чрезмерно большим телом. Если сервер не ограничивает размер принимаемых данных, злоумышленник может отправить десятки или сотни мегабайт информации, что приведёт к исчерпанию оперативной памяти сервера и невозможности обработки запросов других клиентов.

Защита реализована путём ограничения максимального размера тела запроса до одного мегабайта. При попытке передать больший объём данных сервер отклоняет запрос и возвращает код ошибки 413 (Payload Too Large).

На рисунке 2.2 представлен результат работы защиты: сервер отклонил попытку отправить 10 МБ данных.

```
velebes@arch ... (main)> go run cmd/server/main.go
Attack prevented. Error: http: request body too large
```

```
velebes@arch ... (main)> go run cmd/test_large_payload/main.go
Starting Large Payload Test
Server responded with status: 413 Request Entity Too Large
velebes@arch ... (main)>
```

Рисунок 2.2 – Результат работы защиты от атаки большими объёмами данных

2.3 Защита от атаки превышения частоты запросов

Данный тип атаки заключается в массовой отправке большого количества запросов с одного IP-адреса за короткий промежуток времени. Это создаёт высокую нагрузку на вычислительные ресурсы сервера и может привести к замедлению работы или отказу в обслуживании для остальных пользователей.

Для защиты используется механизм ограничения частоты запросов. Для каждого IP-адреса сервер отслеживает количество отправленных запросов и ограничивает скорость до 2 запросов в секунду. При этом допускается кратковременное превышение этого лимита до 5 запросов подряд. При превышении установленного лимита сервер возвращает код ошибки 429 (Too Many Requests).

На рисунке 2.3 представлен результат работы защиты: первые 5 запросов были обработаны успешно, а последующие запросы были отклонены сервером.

```
velebes@arch ... (main)> go run cmd/server/main.go
Connection established
Connection established
Connection established
Connection established
Connection established
```

```
velebes@arch ... (main)> go run cmd/test_rate_limit/main.go
Starting Rate Limit Test
Response 1: Status code 200 (200 OK)
Response 2: Status code 200 (200 OK)
Response 3: Status code 200 (200 OK)
Response 4: Status code 200 (200 OK)
Response 5: Status code 200 (200 OK)
Response 6: Status code 429 (429 Too Many Requests)
Response 7: Status code 429 (429 Too Many Requests)
Response 8: Status code 429 (429 Too Many Requests)
Response 9: Status code 429 (429 Too Many Requests)
Response 10: Status code 429 (429 Too Many Requests)
```

Рисунок 2.3 – Результат работы защиты от атаки превышения частоты запросов

ВЫВОД

В ходе выполнения лабораторной работы разработан защищённый HTTP-сервер с тремя механизмами защиты от атак типа отказ в обслуживании. Защита от Slowloris реализована посредством настройки временных ограничений на уровне сервера, что позволяет автоматически разрывать соединения, не передающие данные в установленные сроки. Защита от атаки большими объёмами данных ограничивает максимальный размер тела запроса одним мегабайтом. Защита от превышения частоты запросов использует механизм ограничения скорости, позволяя каждому клиенту отправлять ограниченное количество запросов в единицу времени. Тестирование подтвердило эффективность всех трёх механизмов защиты.

ПРИЛОЖЕНИЕ А
(Обязательное)
Листинг программного кода сервера

Листинг А.1 – Основной код сервера

```
package main

import (
  "fmt"
  "io"
  "net"
  "net/http"
  "sync"
  "time"

  "golang.org/x/time/rate"
)

var (
  visitors = make(map[string]*rate.Limiter)
  mu       sync.Mutex
)

func getVisitor(ip string) *rate.Limiter {
  mu.Lock()
  defer mu.Unlock()

  limiter, exists := visitors[ip]
  if !exists {
    limiter = rate.NewLimiter(2, 5)
    visitors[ip] = limiter
  }

  return limiter
}

func secureMiddleware(next http.Handler) http.Handler {
  return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
    host, _, err := net.SplitHostPort(r.RemoteAddr)
    if err != nil {
                        http.Error(w,    "Internal    Server    Error",
http.StatusInternalServerError)
      return
    }

    limiter := getVisitor(host)
    if !limiter.Allow() {
      http.Error(w, "Too many requests", http.StatusTooManyRequests)
      return
    }

    r.Body = http.MaxBytesReader(w, r.Body, 1<<20)

    next.ServeHTTP(w, r)
```

```
  })
}

func handler(w http.ResponseWriter, r *http.Request) {
  host, port, _ := net.SplitHostPort(r.RemoteAddr)

  _, err := io.ReadAll(r.Body)
  if err != nil {

    fmt.Printf("Connection dropped for %s. Reason: %v\n", host, err)
    http.Error(w, "Bad Request", http.StatusBadRequest)
    return
  }
  fmt.Printf("Connection established for %s:%s\n", host, port)
}

func readBodyHandler(w http.ResponseWriter, r *http.Request) {
  _, err := io.ReadAll(r.Body)
  if err != nil {

    fmt.Printf("Attack prevented. Error: %v\n", err)
   http.Error(w, "Payload too large", http.StatusRequestEntityTooLarge)
    return
  }

  fmt.Printf("Request was processed successfully\n")
}

func main() {
  mux := http.NewServeMux()
  mux.HandleFunc("/", handler)
  mux.HandleFunc("/readBody", readBodyHandler)

  secureHandler := secureMiddleware(mux)

  server := &http.Server{
    Addr:         ":8089",
    Handler:      secureHandler,
    ReadTimeout:  5 * time.Second,
    WriteTimeout: 10 * time.Second,
    IdleTimeout:  15 * time.Second,
  }

  server.ListenAndServe()
}
```

Листинг А.2 – Тест атаки Slowloris

```
package main

import (
  "bufio"
  "fmt"
  "io"
  "net"
  "sync"
```

```
  "time"
)

const numConnections = 5

func attack(id int, wg *sync.WaitGroup) {
  defer wg.Done()

  conn, err := net.Dial("tcp", "localhost:8089")
  if err != nil {
    fmt.Printf("[Client %d] Connection error: %v\n", id, err)
    return
  }
  defer conn.Close()

  fmt.Fprintf(conn, "POST / HTTP/1.1\r\n")
  fmt.Fprintf(conn, "Host: localhost:8080\r\n")
  fmt.Fprintf(conn, "Content-Length: 15\r\n") // promise 15 bytes
  fmt.Fprintf(conn, "\r\n")

  // send 5 bytes
  fmt.Fprintf(conn, "12345")
  fmt.Printf("[Client %d] Connected, sent partial data ...\n", id)

  start := time.Now()

  _, _ = io.ReadAll(bufio.NewReader(conn))
  duration := time.Since(start)

   fmt.Printf("[Client %d] Server dropped the connection after %.5f
seconds\n", id, duration.Seconds())
}

func main() {
  fmt.Printf("Starting Slowloris attack\n\n")

  var wg sync.WaitGroup

  for i := 1; i <= numConnections; i++ {
    wg.Add(1)
    go attack(i, &wg)
  }

  wg.Wait()

  fmt.Println("\nTest completed. The server successfully defended against
hanging sockets.")
}
```

Листинг А.3 – Тест атаки большими объёмами данных

```
package main

import (
  "bytes"
  "fmt"
```

```
  "net/http"
)

func main() {
  fmt.Println("Starting Large Payload Test")

  largeData := make([]byte, 10*1024*1024)

  resp, err := http.Post("http://localhost:8089/readBody", "application/
octet-stream", bytes.NewReader(largeData))
  if err != nil {
    fmt.Printf("Attack stopped by server. Error: %v\n", err)
    return
  }
  defer resp.Body.Close()

  fmt.Printf("Server responded with status: %s\n", resp.Status)
}
```

Листинг А.4 – Тест атаки превышения лимита запросов

```
package main

import (
  "fmt"
  "net/http"
)

func main() {
  fmt.Println("Starting Rate Limit Test")

  for i := 1; i <= 10; i++ {
    resp, err := http.Get("http://localhost:8089/")
    if err != nil {
      fmt.Printf("Response %d: Error: %v\n", i, err)
      continue
    }

   fmt.Printf("Response %d: Status code %d (%s) \n", i, resp.StatusCode,
resp.Status)
    resp.Body.Close()
  }
}
```